

端侧大模型权重高效利用的系统设计与 NexOS 上实现方案

一、问题定义与背景

在端侧部署大语言模型时，模型权重通常按层载入内存用于计算，其余层的权重存放于磁盘交换空间。在 Lab 2 实现中，`ai_daemon` 在启动时通过 `load_file()` 将**所有权重文件**一次性读入用户态内存。在模型较小时，内存通常可以满足这种实现方案；但若模型参数量较大，就会遇到内存容量不足的问题，必须采用将部分权重调入内存的实现方案，由此在大语言模型计算过程中引入了 I/O 和其优化问题。

本设计的目标是：在 NexOS 现有架构上，设计一套**权重复用 + DMA 双缓冲流水线 + Raw I/O 绕过文件系统缓存**的协同优化方案，使每次权重加载能服务更多计算工作，平衡 CPU 与 I/O 利用率，最大化推理吞吐率。

二、端侧权重优化系统设计

2.1 DMA 双缓冲循环队列

核心思想：采用 DMA 控制器完成单层权重从磁盘换入内存的过程。具体来说，维护 N 个缓冲区（形成一个**循环队列**），每个缓冲区存储一层的权重。CPU 计算第 L 层时，I/O 同时将第 $L + 1$ 层权重通过 DMA 写入下一个缓冲区。理想情况下，I/O 时间可与计算时间近似相同，从而最大化 CPU 和 I/O 利用率。

本质上，这个循环队列可以抽象成为一个**生产者-消费者模型**，其中 DMA 为生产者（向缓冲区填入新一层的权重），计算线程为消费者（从缓冲区读取当前层权重），对循环队列中每个缓冲区使用一个互斥信号量上锁。

`virtio_disk.c` 中现有的 `virtqueue` 已经具备 DMA 描述符链的能力，但 `virtio_disk_rw()` 是同步的。优化后新增异步接口，提交请求后立即返回，完成后通过中断回调通知。

2.2 Raw I/O 绕过文件系统缓存

在权重加载场景下，**文件系统提供的服务可能反而会对 I/O 性能造成拖累**。具体来说，在现有的文件读取路径 `fileread` $\rightarrow$ `readi_user` $\rightarrow$ `bread` $\rightarrow$ `virtio_disk_rw` 中，`bread()` 会将数据缓存到 `bio.c` 的 30 个 `buf` 中。对于大模型权重读取场景（一次性、大批量、顺序读取），块缓存不仅未起到性能优化作用，还会导致内存的额外浪费。

核心思想：新增系统调用 `sys_async_raw_read`，直接将磁盘扇区数据通过 VirtIO `virtqueue` DMA 传输到用户态预分配的锁页内存中，完全绕过 `bio.c` 缓存层。

2.3 权重复用与多请求批处理

核心思想：现有的 ai_daemon **串行**处理各个请求。优化后，调度器在取出请求时先行检查是否存在多个请求可以共享同一层的权重。这样在每个网络层上并发处理这些请求时可以只加载一次该层权重。

并发度自适应：

生产者-消费者模型的稳定状态是生产者写入缓冲队列的速度等于消费者从缓冲队列读取的速度，在本设计中体现为**层计算时间=层加载时间**。由于加载一层权重到缓冲区所需的 I/O 时间可能存在偶然差异且难以人为控制（但**总体上仍然保持稳定**），CPU 和 I/O 的利用率主要取决于批处理大小（一次处理的并发请求数）。批内请求越多，CPU 完成所有请求在一层上的计算时间越长。为了达到 CPU 和 I/O 利用率的最优值，需要调度器不断追踪在执行某一批请求时循环队列中可用缓冲区数量大小变化情况，据此应用**指数平均算法**给出下一批请求的批量，使 CPU 与 I/O 利用率尽可能维持在最优值附近。

在现有 NexOS 上的具体实现方式如下：

1. **时间测量：**利用现有的 uptime() 系统调用（SYS_uptime=14，返回当前 tick 计数）来测量两个关键时间。ai_daemon 在每层计算前后分别调用 uptime()，差值即为该层的计算耗时 T_compute；在提交异步 I/O 请求和检测到完成之间同样测量，得到 I/O 耗时 T_io。
2. **比率追踪：**维护一个指数移动平均值 $\text{ema_ratio} = 0.8 \times \text{ema_ratio} + 0.2 \times (T_{\text{compute}} / T_{\text{io}})$ ，**反映 CPU 与 I/O 的利用率之比**。ema_ratio > 1 时说明 CPU 执行层计算的时间长于 I/O 调入权重的时间，ema_ratio < 1 时说明 I/O 调入权重的时间长于 CPU 执行层计算的时间。
3. **动态调整：**在每个批次开始前检查 ema_ratio：
 - 若 ema_ratio > 1.5，将批量 batch_size 减 1，最低不低于 1；
 - 若 ema_ratio < 0.7，将批量 batch_size 加 1，最高不超过 AI_NREQ=8；
 - 其余情况下，下一批的批量保持不变。
4. **调度器对接：**当前 ai_service.c 的 ai_worker_get() 每次只出队一个请求。需要新增一个非阻塞的 ai_worker_try_get()，队列为空时立即返回 -1 而不 sleep。ai_daemon 的主循环先用 ai_worker_get() 阻塞等待第一个请求到达，然后用 ai_worker_try_get() 尝试再取出最多 batch_size - 1 个请求填充批次。这样既保证有请求时不会空转，又能在请求不足时自然减少实际并发度。
5. **多请求 KV Cache 内存：**当前 ai_daemon 只有一组 kcache / vcache（daemon_runtime 结构体中的单指针）。批处理需要为每个并发请求分配独立的 KV Cache。daemon_runtime_init() 中已计算了 kv_cache_elems，只需改为分配 batch_size 组 kcache/vcache 数组。每组大小为 kv_cache_elems × sizeof(float)（SmoILM2 约为 5.9MB），8 组共约 47MB（k+v 合计约 94MB）。当内存不够时，就触发下面描述的 KV Cache 换页机制。

KV Cache 换页：

当并发请求的 KV Cache 总量超过分配给请求 KV Cache 的可用内存总量时，需要**把暂时不参与计算的“冷请求”的 KV Cache 迁移到交换分区**，把内存资源让给正在计算的“热请求”，在这些冷请求恢复计算时再将它们的 KV Cache 从磁盘交换分区读回。

在现有 NexOS 上的具体实现方式如下：

1. **内存预算与水位线：** ai_daemon 启动时根据 `sbrk(0)` 获得当前可用堆大小，扣除模型权重和工作区后设定 KV Cache 内存配额 `KV_MEM_BUDGET`。每个请求的 KV Cache 占用 $\text{kv_cache_elems} \times \text{sizeof(float)} \times 2$ (k+v, SmolLM2 约 11.8MB)，因此最大并发数 $\text{max_concurrent} = \text{KV_MEM_BUDGET} / (\text{kv_cache_elems} \times \text{sizeof(float)} \times 2)$ 。当活跃请求的 KV Cache 总量达到 `max_concurrent` 时，触发换页。
2. **冷请求选择：** 调度器维护每个活跃请求的 `access_score`（最近一次参与计算的时间戳，用 `uptime()` 记录）。当需要腾出内存空间，按照 **LRU 算法**选择 `access_score` 最小（即最久没被计算过）的请求作为换出对象。被换出的请求状态从 `DECODE` 切换为 `SUSPENDED`。
3. **换出过程（写盘）**
 - `fd = open("/AI/SWAP/req_<id>.kv", O_CREATE | O_WRONLY)` 创建交换文件
 - `write(fd, kcache[slot], kv_cache_elems × sizeof(float))` 写出 K Cache
 - `write(fd, vcache[slot], kv_cache_elems × sizeof(float))` 写出 V Cache
 - `close(fd)` 关闭文件
 - 在 `kv_swap_meta[req_id]` 中记录文件路径和 `in_ram = 0`
 - `memset(kcache[slot], 0, ...)` 清空 DRAM 中的 KV Cache，释放该槽位
4. **换入过程（读盘）：** 当 `SUSPENDED` 状态的请求被调度器选中恢复计算时：
 - 分配一个空闲的 KV Cache 槽位
 - `fd = open("/AI/SWAP/req_<id>.kv", O_RDONLY)` 打开交换文件
 - `read(fd, kcache[slot], kv_cache_elems × sizeof(float))` 读回 K Cache
 - `read(fd, vcache[slot], kv_cache_elems × sizeof(float))` 读回 V Cache
 - `close(fd)` 后 `unlink("/AI/SWAP/req_<id>.kv")` 删除交换文件
 - 更新 `kv_swap_meta[req_id].in_ram = 1`，请求状态切回 `DECODE`
5. **与批处理调度的协同：** 调度器在每个批次结束后的**批次间隙**检查内存水位。若超过高水位，立即换出最冷的请求；若低于低水位，尝试换入一个处于 `SUSPENDED` 状态的请求。

三、关键数据结构与伪代码

3.1 新增数据结构

```
// ===== 内核侧 (kernel/core/ai_service.c 或新文件) =====

// 层组元数据: 描述一次预取操作需要加载的权重范围
struct layer_group_spec {
    int layer_start;           // 起始层号
    int layer_end;             // 结束层号(不含)
    uint64 disk_lba_offset;    // 权重在磁盘上的起始LBA扇区
    uint64 total_bytes;        // 本次传输总字节数
    int buf_slot;              // 目标缓冲区槽位
};

// Raw I/O 请求块: 在内核与驱动间传递
struct raw_io_block {
    uint64 target_paddr;       // 目标锁页内存物理地址
    uint64 disk_lba;           // 磁盘LBA扇区号
    uint64 transfer_size;      // 传输字节数(必须为块大小整数倍)
    int completion_chan;       // 完成通知通道(sleep/wakeup的chan)
    int status;                 // 0=进行中, 1=完成, -1=失败
};

// 锁页内存句柄
struct pinned_mem_handle {
    uint64 vaddr;               // 用户态虚拟地址
    uint64 paddr;               // 物理地址(告知DMA用)
    uint64 size;                // 大小
    int ref_count;              // 引用计数
};

// ===== 用户侧 (user/ai_daemon.c 扩展) =====

// 循环队列缓冲区: 承载DMA写入的锁页内存
struct ring_buffer_io {
    int head;                   // 消费者读取位置
    int tail;                   // 生产者写入位置
    int n_slots;                // 缓冲区数量(推荐2=双缓冲)
    struct pinned_mem_handle *slots; // 各槽位的锁页内存
    int *slot_status;           // 各槽位状态: 0=空闲 1=加载中 2=就绪 3=消费中
    int layer_idx_per_slot[8]; // 记录每个槽位当前存储的层号
};
```

```

// 批处理请求组
struct batch_group {
    int req_ids[8];           // 批内请求ID列表
    int req_positions[8];     // 每个请求的当前计算位置(pos)
    int n_reqs;               // 批内请求数
    int current_layer;        // 当前计算到的层号
    int total_layers;         // 总层数(如SmolLM2=30)
    float *kcache[8];         // 每个请求独立的 K Cache 指针
    float *vcache[8];         // 每个请求独立的 V Cache 指针
    struct ring_buffer_io *rbuf; // 关联的循环队列
};

// KV Cache 换页元数据
struct kv_swap_meta {
    int req_id;               // 所属请求
    int slot_index;           // 当前 KV Cache 槽位索引
    int in_ram;               // 1=在内存中, 0=已换出
    float access_score;       // 访问热度/时间戳(用于 LRU 淘汰决策)
    char filepath[64];        // 换出时的交换文件路径(文件方案)
    uint64 disk_lba;          // 换出时的磁盘 LBA (Raw I/O 方案)
};

```

3.2 新增系统调用

```

// kernel/include/syscall.h 新增
#define SYS_alloc_pinned    29 // 分配锁页内存
#define SYS_free_pinned     30 // 释放锁页内存
#define SYS_async_raw_read  31 // 异步Raw读
#define SYS_async_raw_write 32 // 异步Raw写(用于KV Cache落盘)
#define SYS_poll_io_done    33 // 轮询/等待异步IO完成

// user/user.h 新增用户态接口
int alloc_pinned(uint64 size, uint64 alignment, struct pinned_mem_handle *out);
int free_pinned(struct pinned_mem_handle *handle);
int async_raw_read(uint64 paddr, uint64 disk_lba, uint64 size, int chan_id);
int async_raw_write(uint64 paddr, uint64 disk_lba, uint64 size, int chan_id);
int poll_io_done(int chan_id); // 阻塞等待指定IO完成

```

3.3 核心伪代码

3.3.1 内核：异步 Raw I/O 实现

```
// kernel/core/raw_io.c (新增文件)

// 异步 Raw 读：绕过 bio.c 缓存，直接提交 virtio 请求
// 注：struct buf 的 data 字段为内联数组 uchar data[BSIZE]，不可被重新赋值。
// 本设计中的完整实现要求修改 virtio 驱动使描述符 addr 直接指向用户态锁页内存
// 物理地址（见 4.2 真零拷贝 DMA）；当前 NexOS 的软件模拟方案为：
// 先通过 virtio_disk_rw 将数据 DMA 到内核临时 buf，再 copyout 到用户态，
// 以此绕过 bcache 但保留一次内核→用户拷贝。
int sys_async_raw_read(uint64 paddr, uint64 disk_lba, uint64 size, int chan_id) {
    struct proc *p = myproc();
    // 校验：paddr 必须页对齐、size 必须为 BSIZE 整数倍
    if (paddr % PGSIZE != 0 || size % BSIZE != 0) return -1;

    // 分配内核临时缓冲区（绕过 bcache，不走 bread/brelse）
    uchar *kbuf = kalloc();
    if (kbuf == 0) return -1;
    memset(kbuf, 0, BSIZE);

    // 构造一个临时 buf 结构，不走 bcache，直接提交 virtio 请求
    struct buf tmp;
    memset(&tmp, 0, sizeof(tmp));
    tmp.blockno = disk_lba / (BSIZE / 512);
    // tmp.data 为内联数组，DMA 目标为 kbuf；
    // 未来真零拷贝方案中可将 DMA 目标直接设为用户态锁页物理地址
    memmove(tmp.data, kbuf, BSIZE);

    // 提交到 virtio virtqueue，但不阻塞等待
    // 需要 virtio_disk.c 新增异步提交接口 virtio_disk_submit_async()
    int desc_idx = virtio_disk_submit_async(&tmp, 0 /*read*/);
    if (desc_idx < 0) { kfree(kbuf); return -1; }

    // 注册完成回调：中断到来时 wakeup(chan_id)
    register_io_completion(desc_idx, chan_id);

    // 完成回调中负责：DMA 完成后将 kbuf 内容 copyout 到 paddr 对应的用户虚拟地址，
    // 然后 kfree(kbuf)。（软件模拟阶段存在一次内核→用户拷贝）
    return 0; // 立即返回，不阻塞
}

// virtio_disk.c 异步接口扩展
int virtio_disk_submit_async(struct buf *b, int write) {
```

```

acquire(&disk.vdisk_lock);
int idx[3];
// 分配描述符(可能需要等待)
while (alloc3_desc(idx) < 0) {
    sleep(&disk.free[0], &disk.vdisk_lock);
}
// 组装 virtio_blk_req 并提交到 avail ring
// ... (与现有 virtio_disk_rw 相同的描述符链构建逻辑)
disk.info[idx[0]].b = b;
b->disk = 1;
// 通知设备
*R(VIRTIO_MMIO_QUEUE_NOTIFY) = 0;
release(&disk.vdisk_lock);
return idx[0]; // 返回描述符索引用于完成追踪
}

```

3.3.2 内核：锁页内存分配

```
// kernel/core/vm.c 扩展

// 分配物理连续、页对齐、不可换出的内存
int sys_alloc_pinned(uint64 size, uint64 alignment, uint64 handle_uva) {
    struct proc *p = myproc();
    uint64 npages = size / PGSIZE;
    if (size % PGSIZE != 0) npages++;

    // 在用户地址空间分配连续虚拟页
    uint64 vaddr = p->sz;
    p->sz += npages * PGSIZE;

    // 分配物理页并映射（标记 PTE_D=不可换出，NexOS当前无swap但预留）
    for (uint64 i = 0; i < npages; i++) {
        void *mem = kalloc();
        if (mem == 0) { /* 回滚已分配页 */ return -1; }
        memset(mem, 0, PGSIZE);
        if (mappages(p->pagetable, vaddr + i*PGSIZE, PGSIZE,
                     (uint64)mem, PTE_R | PTE_W | PTE_U) != 0) {
            kfree(mem);
            return -1;
        }
    }
}

// 构造 handle 返回给用户态
struct pinned_mem_handle handle;
handle.vaddr = vaddr;
handle.paddr = walkaddr(p->pagetable, vaddr); // 获取物理地址
handle.size = npages * PGSIZE;
handle.ref_count = 1;

// copyout 到用户空间
if (copyout(p->pagetable, handle_uva, (char *)&handle, sizeof(handle)) < 0)
    return -1;
return 0;
}
```

3.3.3 用户态：ai_daemon 权重管理器、批处理调度与并发度自适应

```
// user/ai_daemon.c 扩展

// ===== 全局自适应调度状态 =====
static int g_batch_size = 1;           // 当前推荐批量大小（1~AI_NREQ）
static float g_ema_ratio = 1.0f;       // T_compute / T_io 的指数移动平均
static int g_max_concurrent = 0;       // 可同时驻留内存的最大请求数
static int g_active_reqs = 0;          // 当前活跃请求数
static int g_kv_mem_used = 0;          // 当前 KV Cache 总内存用量（字节近似）

// ===== 内存预算初始化 =====
// 在 ai_daemon 启动阶段调用，计算 KV Cache 内存预算
static void init_kv_memory_budget(struct daemon_runtime *dr) {
    uint64 heap_top = (uint64)sbrk(0);           // 当前堆顶
    uint64 model_weight_bytes = estimate_model_size(&dr->rt);
    uint64 avail = heap_top - model_weight_bytes; // 扣除模型权重后的可用内存
    g_max_concurrent = (int)(avail / (dr->kv_cache_elems * sizeof(float) * 2));
    if (g_max_concurrent < 1) g_max_concurrent = 1;
    if (g_max_concurrent > AI_NREQ) g_max_concurrent = AI_NREQ;
}

// ===== 循环队列（含互斥信号量） =====
// 每个缓冲区使用互斥信号量上锁
static int weight_ringbuffer_init(struct daemon_runtime *dr, int n_slots) {
    struct ring_buffer_io *rb = &dr->weight_rbuf;
    rb->n_slots = n_slots;
    rb->head = 0;
    rb->tail = 0;
    uint64 layer_bytes = compute_layer_bytes(&dr->rt.cfg);

    for (int i = 0; i < n_slots; i++) {
        if (alloc_pinned(layer_bytes, PGSIZE, &rb->slots[i]) < 0)
            return -1;
        rb->slot_status[i] = 0;           // 0=空闲
        rb->layer_idx_per_slot[i] = -1;
    }
    return 0;
}

// 生产者：异步加载指定层权重到指定缓冲区槽位
// 提交后立即返回，不阻塞
static int prefetch_layer_async(struct daemon_runtime *dr, int layer_idx, int slot) {
    struct ring_buffer_io *rb = &dr->weight_rbuf;
    uint64 lba = compute_layer_lba(&dr->rt.cfg, layer_idx);
```

```

uint64 bytes = compute_layer_bytes(&dr->rt.cfg);
uint64 paddr = rb->slots[slot].paddr;

// 互斥信号量—只有空闲槽位才能被写入
if (rb->slot_status[slot] != 0) return -1;
rb->slot_status[slot] = 1;           // 1=加载中 (DMA 生产者持有)
rb->layer_idx_per_slot[slot] = layer_idx;

// 提交异步 Raw I/O, 立即返回
int chan_id = layer_idx;
if (async_raw_read(paddr, lba, bytes, chan_id) < 0)
    return -1;
return 0;
}

// 消费者: 等待指定槽位就绪并执行批量计算
// 对批内所有请求, 只加载一次该层权重
static int compute_layer_batch(struct daemon_runtime *dr, int slot,
                              struct batch_group *bg) {
    struct ring_buffer_io *rb = &dr->weight_rbuf;
    int layer = rb->layer_idx_per_slot[slot];

    // 阻塞等待 DMA 完成 (中断回调会 wakeup(layer))
    poll_io_done(layer);
    rb->slot_status[slot] = 3;           // 3=消费中 (计算线程消费者持有)

    // 只加载一次该层权重
    float *weight_ptr = (float *)rb->slots[slot].vaddr;
    parse_layer_from_memory(&dr->rt.layers[layer], (char *)weight_ptr,
                           compute_layer_bytes(&dr->rt.cfg),
                           &dr->rt.cfg, dr->rt.layer_kinds[layer]);

    // 对批内所有请求复用同一份权重
    for (int i = 0; i < bg->n_reqs; i++) {
        // 每个请求使用自己独立的 kcache/vcache
        llm_apply_full_layer(&dr->rt, bg->kcache[i], bg->vcache[i], layer,
                            bg->req_positions[i], &dr->ws);
        llm_apply_ffn(&dr->rt.cfg, &dr->rt.layers[layer], &dr->ws);
    }

    rb->slot_status[slot] = 0;           // 释放
    return 0;
}

// ===== ai_service 扩展: 非阻塞出队 (调度器对接) =====
// ai_worker_try_get()—队列空时立即返回-1, 不sleep

```

```

// 需在 kernel/core/ai_service.c 中实现:
// int ai_service_worker_try_get(uint64 token_uva, int cap, uint64 reqid_uva,
//                               uint64 predict_uva) {
//     acquire(&aisvc.lock);
//     if (!aisvc.worker_online || p->pid != aiscv.worker_pid || aiscv.qcount == 0) {
//         release(&aisvc.lock);
//         return -1;           // ★ 不 sleep, 立即返回
//     }
//     int slot = aiscv.q[aisvc.qhead];
//     aiscv.qhead = (aisvc.qhead + 1) % AI_NREQ;
//     aiscv.qcount--;
//     ... // 同 ai_worker_get 的出队 + copyout 逻辑
//     return token_count;
// }

```

// 用户态包装:

```

static int ai_worker_try_get(uint32 *tokens, int cap, int *reqid, int *predict) {
    return (int)__syscall4(SYS_ai_worker_try_get, (long)tokens, cap,
                          (long)reqid, (long)predict);
}

```

// ===== 批次组装 =====

// 先用 ai_worker_get 阻塞等第一个请求,
// 再用 ai_worker_try_get 填满 batch_size-1 个

```

static int assemble_batch(struct daemon_runtime *dr, struct batch_group *bg) {
    memset(bg, 0, sizeof(*bg));

```

// 阻塞等待第一个请求 (保证至少有一个)

```

int n = ai_worker_get(dr->req_tokens[0], AI_DAEMON_TOKEN_MAX,
                     &bg->req_ids[0], &dr->req_predict[0]);

```

```

if (n <= 0) return -1;

```

```

bg->req_positions[0] = 0;

```

// 为第一个请求分配独立 kcache/vcache

```

bg->kcache[0] = dr->kcache_pool[0];

```

```

bg->vcache[0] = dr->vcache_pool[0];

```

```

bg->n_reqs = 1;

```

// 非阻塞尝试填满至 g_batch_size

```

for (int i = 1; i < g_batch_size; i++) {

```

```

    n = ai_worker_try_get(dr->req_tokens[i], AI_DAEMON_TOKEN_MAX,
                        &bg->req_ids[i], &dr->req_predict[i]);

```

```

    if (n <= 0) break;           // 队列已空, 当前批次结束

```

```

    bg->req_positions[i] = 0;

```

```

    bg->kcache[i] = dr->kcache_pool[i];

```

```

    bg->vcache[i] = dr->vcache_pool[i];

```

```

    bg->n_reqs++;

```

```

}

// 更新活跃请求计数
g_active_reqs += bg->n_reqs;
g_kv_mem_used += bg->n_reqs * (int)(dr->kv_cache_elems * sizeof(float) * 2);

bg->current_layer = 0;
bg->total_layers = dr->rt.cfg.n_layers;
bg->rbuf = &dr->weight_rbuf;
return 0;
}

// ===== 自适应批量调整 =====
static void adapt_batch_size(int compute_ticks, int io_ticks) {
    if (io_ticks <= 0) return; // 避免除零
    float ratio = (float)compute_ticks / (float)io_ticks;
    // 指数移动平均
    g_ema_ratio = 0.8f * g_ema_ratio + 0.2f * ratio;

    // 动态调整
    if (g_ema_ratio > 1.5f && g_batch_size > 1) {
        g_batch_size--;
    } else if (g_ema_ratio < 0.7f && g_batch_size < AI_NREQ) {
        g_batch_size++;
    }
    // 其余情况保持不变
}

// ===== 批处理调度主循环（含时间测量与自适应） =====
static int run_batched_inference(struct daemon_runtime *dr,
                                struct batch_group *bg) {
    int n_layers = dr->rt.cfg.n_layers;
    struct ring_buffer_io *rb = &dr->weight_rbuf;
    int n_slots = rb->n_slots;
    int total_compute_ticks = 0, total_io_ticks = 0;

    // 预取前 n_slots 层
    for (int i = 0; i < n_slots && i < n_layers; i++) {
        prefetch_layer_async(dr, i, i);
    }

    // 流水线执行
    for (int layer = 0; layer < n_layers; layer++) {
        int slot = layer % n_slots;

        // 测量计算时间

```

```

    int t1 = uptime();
    compute_layer_batch(dr, slot, bg);
    int t2 = uptime();
    total_compute_ticks += (t2 - t1);

    // 测量 I/O 时间（异步提交 → 等待完成）
    int next_layer = layer + n_slots;
    if (next_layer < n_layers) {
        int t3 = uptime();
        prefetch_layer_async(dr, next_layer, slot);
        // 不在此处 poll——下一轮 compute_layer_batch 的层号不同，
        // 当前层的 I/O 是预取"将来"的层，真正的 I/O 等待发生在
        // 对应层被消费时的 poll_io_done() 内。
        // 这里记录的是本轮 I/O 提交耗时（异步，几乎为 0）
        int t4 = uptime();
        total_io_ticks += (t4 - t3);
    }
}

// 依据本轮测量结果调整下一批的大小
adapt_batch_size(total_compute_ticks, total_io_ticks);

// 更新活跃请求计数和内存用量
g_active_reqs -= bg->n_reqs;
g_kv_mem_used -= bg->n_reqs * (int)(dr->kv_cache_elems * sizeof(float) * 2);

// 批次间隙检查内存水位
check_watermark_and_swap(dr);

return 0;
}

// 批次间隙的水位检查
static void check_watermark_and_swap(struct daemon_runtime *dr) {
    if (g_active_reqs >= g_max_concurrent) {
        // 超过高水位：选出最冷的请求换出
        int cold_id = select_coldest_request(dr);
        if (cold_id >= 0)
            kv_cache_swap_out(dr, cold_id);
    } else if (g_active_reqs < g_max_concurrent / 2 && has_suspended_request(dr)) {
        // 低于低水位：换入一个挂起请求
        int next_id = select_next_suspended(dr);
        if (next_id >= 0)
            kv_cache_swap_in(dr, next_id);
    }
}

```

}
}

3.3.4 KV Cache 换页（文件系统方案 + LRU 淘汰）

```
// user/ai_daemon.c - KV Cache 换页

// ===== 辅助：选择最冷的请求（LRU） =====
// 按 access_score 选择最久没使用的请求
static int select_coldest_request(struct daemon_runtime *dr) {
    int coldest_id = -1;
    float min_score = (float)(1 << 30);
    for (int i = 0; i < AI_NREQ; i++) {
        struct kv_swap_meta *m = &dr->kv_swap_table[i];
        if (m->in_ram && m->access_score < min_score && m->req_id > 0) {
            min_score = m->access_score;
            coldest_id = m->req_id;
        }
    }
    return coldest_id;    // 返回最久未访问的请求 id
}

// 每个批次计算后更新 access_score
static void update_access_score(struct daemon_runtime *dr, int req_id) {
    struct kv_swap_meta *m = find_kv_meta(dr, req_id);
    if (m != NULL) {
        m->access_score = (float)uptime();    // 记录最近一次访问时间戳
    }
}

// ===== 换出：LRU 选择 + 文件写盘 =====
static int kv_cache_swap_out(struct daemon_runtime *dr, int req_id) {
    struct kv_swap_meta *meta = find_kv_meta(dr, req_id);
    if (meta == NULL || !meta->in_ram) return -1;

    int slot = meta->slot_index;
    uint64 kv_bytes = dr->kv_cache_elems * sizeof(float);
    char path[64];

    // 创建交换文件
    snprintf(path, sizeof(path), "/AI/SWAP/req_%d.kv", req_id);
    int fd = open(path, O_CREATE | O_WRONLY);
    if (fd < 0) return -1;

    // 写出 K Cache → V Cache
    if (write(fd, (void *)dr->kcache_pool[slot], (int)kv_bytes) < 0) goto fail;
    if (write(fd, (void *)dr->vcache_pool[slot], (int)kv_bytes) < 0) goto fail;
    close(fd);
fail:
}
```

```

// 更新元数据
strcpy(meta->filepath, path);
meta->in_ram = 0;

// 清空 DRAM 中的 KV Cache
memset(dr->kcache_pool[slot], 0, (uint)kv_bytes);
memset(dr->vcache_pool[slot], 0, (uint)kv_bytes);

g_active_reqs--;
g_kv_mem_used -= (int)(kv_bytes * 2);
return 0;

fail:
    close(fd);
    unlink(path);
    return -1;
}

// ===== 换入：文件读盘 + 恢复 =====
static int kv_cache_swap_in(struct daemon_runtime *dr, int req_id) {
    struct kv_swap_meta *meta = find_kv_meta(dr, req_id);
    if (meta == NULL || meta->in_ram) return -1;

    uint64 kv_bytes = dr->kv_cache_elems * sizeof(float);

    // 分配空闲槽位
    int slot = find_free_kv_slot(dr);
    if (slot < 0) {
        // 所有槽位都被占用，先换出最冷的
        int cold_id = select_coldest_request(dr);
        if (cold_id < 0 || kv_cache_swap_out(dr, cold_id) < 0) return -1;
        slot = find_free_kv_slot(dr);
    }

    // 打开交换文件
    char path[64];
    snprintf(path, sizeof(path), "/AI/SWAP/req_%d.kv", req_id);
    int fd = open(path, O_RDONLY);
    if (fd < 0) return -1;

    // 读回 K Cache → V Cache
    int r1 = read(fd, (void *)dr->kcache_pool[slot], (int)kv_bytes);
    int r2 = read(fd, (void *)dr->vcache_pool[slot], (int)kv_bytes);
    if (r1 < 0 || r2 < 0) { close(fd); return -1; }
}

```

```

// 关闭并删除交换文件
close(fd);
unlink(path);

// 更新元数据
meta->in_ram = 1;
meta->slot_index = slot;
meta->filepath[0] = '\0';

g_active_reqs++;
g_kv_mem_used += (int)(kv_bytes * 2);
return 0;
}

// ===== 查询是否有挂起的请求（辅助 low-watermark 判断） =====
static int has_suspended_request(struct daemon_runtime *dr) {
    // 遍历 kv_swap_table，检查是否存在 in_ram==0 的条目
    for (int i = 0; i < AI_NREQ; i++) {
        if (!dr->kv_swap_table[i].in_ram && dr->kv_swap_table[i].req_id > 0)
            return 1;
    }
    return 0;
}

```

当 `async_raw_write` 和专用交换磁盘分区可用后，可将 `open/write` 替换为 `async_raw_write`，`open/read` 替换为 `async_raw_read`，从而绕过 `bio.c` 避免缓存污染。

3.3.5 内核：virtio_disk 中断回调（异步完成通知）

修改 `virtio_disk_isr()` 和 `complete_used_locked()`，使其在完成描述符链后唤醒指定的 `chan`，而非通用 `b` 指针。

```

// kernel/drivers/virtio_disk.c 扩展

// 新增: I/O 完成追踪表
struct io_completion {
    int desc_idx;        // 描述符链首索引
    int chan_id;         // 完成通知通道 (用于 wakeup)
    int active;          // 1=等待中
} io_completions[NUM]; // NUM=8, 与 virtqueue 深度一致

// 注册完成回调—异步提交时调用
void register_io_completion(int desc_idx, int chan_id) {
    for (int i = 0; i < NUM; i++) {
        if (!io_completions[i].active) {
            io_completions[i].desc_idx = desc_idx;
            io_completions[i].chan_id = chan_id;
            io_completions[i].active = 1;
            return;
        }
    }
}

// 中断回调通知
void virtio_disk_isr(void) {
    acquire(&disk.vdisk_lock);
    *R(VIRTIO_MMIO_INTERRUPT_ACK) = *R(VIRTIO_MMIO_INTERRUPT_STATUS) & 0x3;
    __sync_synchronize();
    complete_used_locked(); // 现有逻辑: 释放完成描述符
    release(&disk.vdisk_lock);

    // 额外遍历 io_completions 表, wakeup 对应的 chan
    acquire(&disk.vdisk_lock);
    for (int i = 0; i < NUM; i++) {
        if (io_completions[i].active && disk.info[i].status == 0) {
            wakeup((void *) (uintptr_t) io_completions[i].chan_id);
            io_completions[i].active = 0;
        }
    }
    release(&disk.vdisk_lock);
}

// poll_io_done 内核实现—阻塞等待指定 chan 被 wakeup
int sys_poll_io_done(int chan_id) {
    acquire(&disk.vdisk_lock);
    while (1) {
        int done = 0;

```

```

    for (int i = 0; i < NUM; i++) {
        if (io_completions[i].chan_id == chan_id && !io_completions[i].active) {
            done = 1; break;
        }
    }
    if (done) break;
    sleep((void *) (uintptr_t)chan_id, &disk.vdisk_lock);
}
release(&disk.vdisk_lock);
return 0;
}

```

在当前 NexOS 上，可以通过同步 `virtio_disk_rw()` + 用户态多线程模拟异步语义来验证上层逻辑。

3.3.6 内核：Raw I/O 版本 KV Cache 换页

`async_raw_read` / `async_raw_write` 可用后的文件 I/O 路径。

// Raw I/O 换出——绕过文件系统和 bio.c, 直接 DMA

```
static int kv_cache_swap_out_raw(struct daemon_runtime *dr, int req_id) {
    struct kv_swap_meta *meta = find_kv_meta(dr, req_id);
    int slot = meta->slot_index;
    uint64 kv_bytes = dr->kv_cache_elems * sizeof(float);

    uint64 swap_lba = alloc_swap_lba(kv_bytes * 2); // k+v = 2×
    // 直接 DMA 到磁盘, 不经过 open/write
    if (async_raw_write(dr->kcache_pool[slot], swap_lba, kv_bytes, req_id) < 0)
        return -1;
    poll_io_done(req_id);

    meta->disk_lba = swap_lba;
    meta->in_ram = 0;
    memset(dr->kcache_pool[slot], 0, (uint)kv_bytes);
    memset(dr->vcache_pool[slot], 0, (uint)kv_bytes);
    return 0;
}
```

// Raw I/O 换入

```
static int kv_cache_swap_in_raw(struct daemon_runtime *dr, int req_id) {
    struct kv_swap_meta *meta = find_kv_meta(dr, req_id);
    int slot = find_free_kv_slot(dr);
    uint64 kv_bytes = dr->kv_cache_elems * sizeof(float);

    if (async_raw_read(dr->kcache_pool[slot], meta->disk_lba, kv_bytes, req_id) < 0)
        return -1;
    poll_io_done(req_id);

    meta->in_ram = 1;
    meta->slot_index = slot;
    return 0;
}
```

四、实现边界分析

4.1 可基于当前 NexOS 实现的部分

模块	可实现内容	备注
请求调度与状态管理	批处理调度器、并发度自适应、请求状态机扩展	ai_service.c 实现 8 槽请求队列和完整状态机，可扩展 batch_group 等字段
权重复用	多请求共享同一次权重前向传播	ai_daemon.c 中 run_decode_steps() 采用按层循环的模式，可改为对多请求批量执行 llm_apply_full_layer
前缀缓存复用	多请求前缀匹配与 KV Cache 复用	ai_daemon.c 实现 daemon_prefix_cache ，可扩展为多请求共享
Raw I/O 旁路 (软件模拟)	新增系统调用绕过 bio.c ，直接调用 virtio_disk_rw	virtio_disk.c 已实现 virtio_disk_rw() ，可构造临时 buf 结构直接调用该函数以绕过 bread/brelse 。在软件模拟阶段，使用同步的 virtio_disk_rw （先通过 DMA 拷贝到内核临时缓冲区，再 copyout 到用户态）；实现异步 I/O 与真零拷贝 DMA 后，可替换为 virtio_disk_submit_async （通过 DMA 直接拷贝到用户态）
双缓冲流水线 (软件模拟)	循环队列结构、生产者-消费者同步、层组预取逻辑	使用 sbrk 分配用户态内存模拟锁页内存，通过 sleep/wakeup 实现同步
KV Cache 换页 (逻辑)	元数据管理、淘汰决策、通过文件系统写入	ai_daemon.c 的 prefix_cache_save_after_prefill 已有 KV Cache 序列化的雏形

模块	可实现内容	备注
系统调用扩展	新增 sys_alloc_pinned、sys_async_raw_read 等	syscall.c 已有清晰的系统调用分发机制， 新增条目即可
页表操作	获取物理地址、建立映射	vm.c 中 walk()、walkaddr()、mappages() 可直接使用

4.2 需要 NexOS 未来进一步支持的部分

模块	限制	需要的支持
真正的异步 I/O	virtio_disk_rw() 当前同步阻塞，提交后 sleep(b) 等待完成	需将 virtio_disk.c 重构为异步提交 + 中断回调模式，virtio_disk_isr() 中 wakeup 指定 chan 而非通用 b
真零拷贝 DMA	当前 virtio 描述符指向 b->data (内核缓存)， 非用户态物理地址	需修改 virtio 驱动，使描述符 addr 直接指向用户态锁页内存物理地址，绕过内核 buf 中间层
物理连续内存分配	kalloc() 分配单页 (4KB)， 无法保证多页连续	需实现连续物理页分配器 (类似 __get_free_pages)，或使用散列-收集 (scatter-gather) 描述符链
锁页内存 (Pinned Memory)	NexOS 无 swap 机制， 所有内存隐式"锁定"， 但无显式标记	虽然功能上等价，但未来若引入 swap 需添加 PTE_D (dirty/non-pageable) 标志位
NPU/Tensor Core 加速	矩阵乘法在 CPU 上用 qmatmul 软件执行， $T_{compute} \gg T_{io}$	需 NPU 驱动支持，否则双缓冲流水线在 CPU 模拟下无法体现"计算掩盖 I/O"效果
磁盘分区管理	当前只有一个 virtio block 设备，无分区概念	需要支持磁盘分区或第二块专用块设备， 用于权重 Raw I/O 与 KV Cache swap 分区
真实 DMA 总线竞争	QEMU 模拟环境下 DMA 与 CPU 共享内存总线的行为与真实 SoC 不同	需真实端侧硬件 (如带 NPU 的 ARM SoC) 验证 UMA 总线竞争效应
SSD 写放大与寿命	QEMU virtio-blk 无 FTL 层， 无法模拟闪存擦写特性	需真实 NVMe/eMMC 设备评估 KV Cache 换页的写放大影响

模块	限制	需要的支持
并发度-CPU/IO 利用率测试	单核 QEMU 环境下无法真实测量 CPU-I/O 重叠	需多核支持或真实硬件， 使用硬件性能计数器测量利用率

五、参考资料

编号	论文/系统	核心借鉴点
1	LLM in a flash (arXiv:2312.11514)	上下文稀疏性预测、滑动窗口内存管理、 行列捆绑布局优化 DMA 传输效率
2	ActiveFlow / Active-Weight Swapping (arXiv:2504.08378)	活跃权重 DRAM-Flash 交换流水线、层组 (Layer Group) 预取机制、锁页内存作为 DMA 目标
3	KVSwap (arXiv:2511.11907)	KV Cache 磁盘换页框架、 基于注意力分数的精准预取与分组淘汰策略
4	HiFC (NeurIPS 2025)	高效 Flash-based KV Cache 交换，写放大效应分析与 SSD 寿命考量
5	Linux O_DIRECT (POSIX Direct I/O)	绕过页缓存的直接 I/O 机制， 用户态缓冲区对齐与大小约束
6	VirtIO Specification	virtqueue 描述符链、可用环/已用环的异步通知机制， 作为 DMA 传输的底层基础
7	NexOS / xv6 教学内核	生产者-消费者请求队列、sleep/wakeup 同步原语、 三级页表管理、块缓冲缓存
8	KVStream (Microsoft, 2025)	端侧 LLM 推理内存管理，KV Cache 流式淘汰策略